

Projecto de Disciplina: Implementação de Problemas de Busca em Inteligência Artificial

1 Informação Geral

Objectivo geral:

Cada grupo deverá implementar, em Python, uma aplicação que resolva um problema complexo de busca em Inteligência Artificial, incluindo uma interface de utilizador que permita configurar o problema, executar algoritmos de busca e visualizar os resultados.

Prazo de entrega: Até antes da Defesa do Trabalho.

Formato de entrega:

Cada grupo deverá entregar a seguinte informação:

- Código-fonte,
- relatório em PDF,
- apresentação e
- demonstração funcional.

2 Bibliotecas recomendadas para interface

Os estudantes podem usar:

Biblioteca	Uso recomendado
Tkinter	Interfaces simples de desktop. É a interface gráfica padrão do Python e está disponível na maioria das instalações.
PySide6 / PyQt6	Interfaces desktop mais profissionais e complexas. PySide6 é a ligação oficial do Qt para Python.
Kivy	Aplicações com interface moderna, touch ou possibilidade de correr em desktop/mobile. Suporta Windows, Linux, macOS, Android e iOS.
Pygame	Visualizações gráficas, jogos, mapas, labirintos e simulações.
Streamlit	Interface web simples para demonstrar algoritmos e dashboards interactivos.

3 Responsabilidades dos membros por grupo

Sugere-se quem em cada grupo, cada um dos elementos consiga mostrar dominio de pelo menos 3 das seguintes competencias seja:

1. **Modelador do Problema:** define estados, acções, estado inicial, objectivo e custo.

2. **Programador dos Algoritmos:** implementa BFS, DFS, UCS, IDS, Gulosa, A* ou variantes exigidas.
3. **Desenvolvedor da Interface:** cria a interface gráfica ou web.
4. **Responsável por Visualização e Métricas:** mostra caminhos, nós expandidos, custo, tempo e memória.
5. **Testes, Relatório e Apresentação:** valida casos de teste, documenta resultados e prepara a defesa.

4 Lista de Grupos e Enunciados

4.1 Lista de Grupos

Grupo	Estudantes	Tema
G1	• Aashir Omar (20240143); • Igor dos Santos (20240459); • Stanley Cossa (20240567); • Quirson Ngale (20240526); • Saudah Ahmad (20220057); • Hélder Júnior (20230414)	• 4.2.8
G2	• Celestino Siteo (20240530); • Jenifa Crish (20240461); • Florizele Beatriz (20240673); • Shontelle Friães (20230963); • Yumna Sultan (20230657); • Yumny Tembe (20240457)	• 4.2.4
G3	• Aaliyah Guevane (20240522); • Milvia Gome (20240536); • Maria Paulina (20240671); • Sofia Amélia (20230410); • Lucília Chihale (20230371); • Edurne Sanfins (20230440)	• 4.2.5
G4	• Bashir Laher (20210311); • Eden Siteo (20231116); • Luís Lopes (20230726); • Saldon Come (20220826); • Torres Tomé (20190308); • Whezo Castanheira (20230401)	• 4.2.1
G5	• Shuraim Hussene Ishakgi (20190461); • Martin Daud Ribeiro (20200039); • Vritik Valabdas (20190025); • Saymara Chambal (20240187) • Shelton Arlindo Chicuava • Rahissa Mahomed Al Yashin	• 4.2.11

G6	• Dikshy Guinés (20240213); • Monardos Macário (20240675); • Vicente Macuacua (20240208); • Luana Machel (20220329); • Nanila Yusra (20240916); • Fernando Pecado (20180967)	• 4.2.2
G7	• Adayl Rafik (20240330); • Cleyton Azize (20240534); • Muhamad Shahin (20240568); • Usseid Adil (20240394) • Ilídio Guissamulo (20221022) • 	• 4.2.9
G8	• Ayko Sambo (20190217); • Gerson Ouana (20240948); • Kenny Matsule (20240908); • Yannis Chilundo (20220156); • Yasin Magno (20240260) • Malik Dauto	• 4.2.10
G9	• Eufates Cossa (20210513); • Shelton Mindo (20220514); • Nelson Mombi (20220866); • Kelvin Sibambo (20221057); • Ecclesio Pembelane (20190509) • Julia Mutimucuo	• 4.2.6
G10	• Maximiano Chiche (20240668); • Francisco Penicela (20240466); • Danila Mateus (20240348); • Floyd Picardo (20240456); • Virgílio João (20240997); • Zahra Muhammad Shoib (20240418)	• 4.2.14
G11	• Ana Maibasse (20180193); • Cristina Almeida (20240535); • Dário Mazuze (20240367); • Neyal Santilal (20240629); • Winacy Vitorino (20230609) • Shenil Gafur	• 4.2.7
G12	• Fahima Samsudin (20220144); • Ranim Amade (20240138); • Isma Cudbudin (20220133); • Laiq Omar (20240309) • Rosina António (20211069); •  Edson Nhacule (20220412);	• 4.2.3

G13	• Caiado Binda (20240873); • Helenio Lucas (20230383); • Laerson Mogas (202106666); • Winner Doho (20241114); • Tânia Bambo (20210335) • Ziyad Luto	• 4.2.12
G14	• Luana Mabota (20220792); • Tiago Correia (20200018); • Keny Muchanga (20210410); • Vlad Nguila (20190134); • Meyvin da Silva (20220281); • Yasser Carimo (20220384)	• 4.2.13

4.2 Enunciados

4.2.1 Robô de Entrega em Campus Universitário

Descrição do Problema

Pretende-se desenvolver um sistema inteligente que simule um robô de entregas autónomo dentro de um campus universitário. O campus deve ser modelado como um grafo ou matriz, onde:

- Cada nó representa um edifício ou ponto de interesse
- As ligações representam caminhos possíveis
- Alguns caminhos podem estar bloqueados
- Alguns caminhos têm custos diferentes (ex: escadas, trânsito pedonal, distância)

O robô deve:

- Sair de um ponto inicial
- Entregar encomendas em um ou mais destinos
- Encontrar o percurso mais eficiente

Requisitos Funcionais

O sistema deve permitir:

- Definir o mapa (manual ou carregado de ficheiro)
- Seleccionar:
 - Ponto de partida
 - Um ou vários destinos

- Escolher o algoritmo de busca
- Visualizar o percurso encontrado

Interface de Utilizador

A interface deve:

- Mostrar o mapa do campus (grid ou grafo)
- Permitir clicar para definir origem e destino(s)
- Mostrar o caminho encontrado visualmente
- Indicar nós explorados durante a busca

Requisitos

- Suporte a múltiplos destinos (problema multi-objectivo)
- Comparação entre algoritmos (tempo, custo, nós expandidos)
- Permitir adicionar/remover obstáculos dinamicamente

Saídas esperadas

- Caminho encontrado
- Custo total
- Número de nós expandidos
- Tempo de execução

4.2.2 Planeamento de Rotas em Maputo com Trânsito

Descrição do Problema

Desenvolver um sistema de navegação que calcule a melhor rota entre dois pontos na cidade de Maputo. O sistema deve considerar:

- Distância entre pontos
- Trânsito (custos variáveis)
- Estradas bloqueadas
- Diferença entre rotas mais curtas vs mais rápidas

O problema deve ser modelado como um grafo ponderado.

Requisitos Funcionais

O sistema deve permitir:

- Definir ou carregar um mapa simplificado da cidade

- Escolher origem e destino
- Definir condições de trânsito:
 - Baixo
 - Médio
 - Alto
- Executar diferentes algoritmos de busca

Interface de Utilizador

A interface deve:

- Mostrar um mapa simplificado da cidade
- Permitir seleccionar pontos clicando
- Mostrar o caminho encontrado
- Visualizar custo acumulado ao longo do percurso

Requisitos

- Permitir alterar dinamicamente o trânsito
- Comparar resultados entre algoritmos
- Implementar heurística realista (ex: distância euclidiana)

Saídas esperadas

- Melhor rota encontrada
- Custo total (tempo ou distância)
- Nós explorados
- Comparação entre algoritmos

4.2.3 Resolução Avançada de Labirintos

Descrição do Problema

Criar um sistema que gere e resolva labirintos automaticamente. O labirinto deve ser representado como:

- Uma matriz (grid)
- Com células livres e bloqueadas
- Com ponto inicial e final

Requisitos Funcionais

O sistema deve permitir:

- Gerar labirintos automaticamente (tamanho variável)
- Inserir manualmente obstáculos
- Definir posição inicial e final
- Executar algoritmos de busca

Interface de Utilizador

A interface deve:

- Mostrar o labirinto graficamente
- Permitir edição do labirinto
- Visualizar a busca passo a passo
- Destacar:
 - Nós visitados
 - Caminho final

Requisitos

- Obstáculos dinâmicos (mudam durante execução)
- Diferentes custos por célula
- Animação da execução dos algoritmos

Saídas esperadas

- Caminho encontrado
- Número de passos
- Nós explorados
- Tempo de execução

4.2.4 Jogo dos 8-Puzzle e 15-Puzzle

Descrição do Problema

Desenvolver uma aplicação que resolva automaticamente os problemas clássicos do **8-Puzzle (3x3)** e **15-Puzzle (4x4)**.

O problema consiste em:

- Um tabuleiro com peças numeradas e um espaço vazio
- Movimentos possíveis: deslizar peças para o espaço vazio
- Objectivo: atingir uma configuração final ordenada

Requisitos Funcionais

O sistema deve permitir:

- Escolher entre 8-Puzzle ou 15-Puzzle
- Inserir manualmente o estado inicial ou gerar aleatoriamente
- Validar se o problema é solucionável
- Executar algoritmos de busca para encontrar a solução

Heurísticas obrigatórias (para A*)

- Número de peças fora do lugar
- Distância de Manhattan

Interface de Utilizador

A interface deve:

- Mostrar o tabuleiro graficamente
- Permitir mover peças manualmente
- Mostrar a solução passo a passo
- Indicar profundidade da solução

Requisitos

- Comparação entre heurísticas (eficiência e qualidade)
- Visualização da árvore de busca (opcional mas valorizado)
- Controlo de velocidade da animação

Saídas esperadas

- Sequência de movimentos
- Número de passos até solução
- Nós expandidos
- Tempo de execução
- Comparação entre heurísticas

4.2.5 Planeamento de Movimento de um Drone

Descrição do Problema

Criar um sistema que simule o movimento de um drone autónomo num ambiente com restrições. O ambiente deve ser modelado como:

- Espaço 2D ou 3D (grid ou grafo)
- Com obstáculos fixos
- Com zonas de risco (custos elevados)
- Com zonas proibidas (não transitáveis)

- drone deve encontrar a melhor rota entre dois pontos.

Requisitos Funcionais

O sistema deve permitir:

- Definir:
 - Ponto inicial
 - Destino
 - Obstáculos
 - Zonas de risco
- Escolher algoritmo de busca
- Executar e visualizar a trajectória

Interface de Utilizador

A interface deve:

- Mostrar o ambiente (grid ou mapa)
- Permitir desenhar obstáculos e zonas de risco
- Visualizar o caminho do drone
- Mostrar o custo acumulado ao longo do percurso

Requisitos

- Suporte a movimento diagonal (ou em 3D)
- Diferentes tipos de custo (distância, risco, energia)
- Comparação entre algoritmos

Saídas esperadas

- Trajectória do drone
- Custo total
- Nós explorados
- Tempo de execução

4.2.6 Sistema de Navegação em Armazém Inteligente

Descrição do Problema

Desenvolver um sistema para um armazém automatizado, onde um agente deve recolher produtos e levá-los até uma zona de saída. O armazém deve conter:

- Prateleiras (obstáculos)
- Corredores (zonas transitáveis)

- Produtos distribuídos em diferentes locais
- Zona de entrega

O problema envolve otimização de percurso com múltiplos pontos de recolha.

Requisitos Funcionais

O sistema deve permitir:

- Definir layout do armazém
- Inserir:
 - Localização dos produtos
 - Ponto inicial do agente
 - Zona de entrega
- Executar algoritmo para determinar a melhor sequência de recolha

Interface de Utilizador

- A interface deve:
- Representar o armazém graficamente
- Mostrar:
 - Agente
 - Produtos
 - Caminhos
- Visualizar a sequência de recolha e entrega

Requisitos

- Otimização da ordem de recolha (tipo problema do caixeiro-viajante simplificado)
- Custos diferentes por zona (ex: corredores estreitos)
- Simulação em tempo real

Saídas esperadas

- Ordem de recolha dos produtos
- Caminho total percorrido
- Custo total
- Nós explorados
- Comparação entre estratégias

4.2.7 Missionários e Canibais Generalizado

Descrição do Problema

Desenvolver uma versão generalizada e configurável do clássico problema dos Missionários e Canibais.

Regras clássicas:

- Existem missionários e canibais numa margem do rio
- Um barco transporta pessoas entre margens
- Em qualquer margem, os canibais não podem ser mais numerosos que os missionários (caso contrário, os missionários são “comidos”)

Nesta versão, o sistema deve permitir:

- Número variável de missionários e canibais
- Capacidade variável do barco
- Regras adicionais opcionais

Modelação esperada

Os estudantes devem definir claramente:

- Representação do estado (ex: (M_esq, C_esq, barco))
- Operadores (movimentos possíveis)
- Teste de objectivo
- Função de custo (se aplicável)

Requisitos Funcionais

O sistema deve permitir:

- Configurar:
 - Número de missionários
 - Número de canibais
 - Capacidade do barco
- Executar algoritmos de busca
- Mostrar a sequência de estados até solução

Interface de Utilizador

A interface deve:

- Mostrar visualmente as duas margens
- Representar missionários e canibais
- Mostrar o movimento do barco
- Exibir cada estado da solução

Requisitos avançados

- Permitir regras adicionais (ex: limite de viagens, custo por travessia)
- Detectar automaticamente estados inválidos
- Mostrar a árvore de busca (opcional)

Saídas esperadas

- Sequência de estados
- Número de passos
- Nós explorados
- Tempo de execução

4.2.8 Planeamento de Viagens com Múltiplas Cidades

Descrição do Problema

Desenvolver um sistema para planear uma viagem entre várias cidades, inspirado no problema do caixeiro-viajante (TSP). O agente deve:

- Partir de uma cidade inicial
- Visitar todas as cidades
- Minimizar custo total (distância, tempo ou custo monetário)

Modelação esperada

- Estados: cidade actual + conjunto de cidades visitadas
- Operadores: viajar para cidade não visitada
- Objectivo: todas cidades visitadas
- Função de custo acumulado

Requisitos Funcionais

O sistema deve permitir:

- Definir número de cidades
- Inserir matriz de distâncias (ou gerar automaticamente)
- Escolher cidade inicial
- Executar algoritmos de busca

Interface de Utilizador

A interface deve:

- Mostrar cidades num plano (2D)

- Representar ligações entre cidades
- Visualizar o percurso final
- Destacar ordem de visita

Requisitos

- Implementar heurística admissível para A*
- Comparar solução óptima vs aproximada
- Suporte a número elevado de cidades (optimização necessária)

Saídas esperadas

- Ordem de visita das cidades
- Custo total
- Nós explorados
- Tempo de execução

4.2.9 Sistema de Evacuação de Emergência

Descrição do Problema

Criar um sistema que simule a evacuação de pessoas de um edifício em situação de emergência (incêndio, desastre, etc.). O edifício deve conter:

- Salas
- Corredores
- Saídas de emergência
- Zonas de perigo (ex: fogo, fumo)

O objectivo é encontrar rotas seguras e eficientes para evacuação.

Modelação esperada

- Estados: posição de uma pessoa no edifício
- Operadores: movimentos possíveis
- Custos:
 - Distância
 - Nível de perigo
- Objectivo: alcançar uma saída segura

Requisitos Funcionais

O sistema deve permitir:

- Definir layout do edifício

- Inserir:
 - Pessoas
 - Saídas
- Zonas de perigo
- Simular evacuação

Interface de Utilizador

A interface deve:

- Representar o edifício (grid ou planta)
- Mostrar pessoas em movimento
- Destacar rotas seguras
- Indicar zonas perigosas

Requisitos

- Evacuação de múltiplas pessoas simultaneamente
- Obstáculos dinâmicos (ex: propagação do fogo)
- Comparação entre estratégias de evacuação

Saídas esperadas

- Caminhos de evacuação
- Tempo total de evacuação
- Número de nós explorados
- Custo associado ao risco

4.2.10 Resolução de Sudoku com Busca

Descrição do Problema

Desenvolver uma aplicação que resolva automaticamente puzzles de **Sudoku 9x9**, utilizando técnicas de busca e satisfação de restrições. O sistema deve receber um tabuleiro parcialmente preenchido e encontrar uma solução válida, respeitando as regras do Sudoku.

Modelação esperada

Os estudantes devem definir:

- Estado: configuração actual do tabuleiro.
- Acções: preencher uma célula vazia com um número válido.
- Teste de objectivo: tabuleiro completo e válido.
- Restrições: linhas, colunas e subgrades 3x3 sem números repetidos.

Requisitos Funcionais

O sistema deve permitir:

- Inserir manualmente um Sudoku.
- Carregar exemplos pré-definidos.
- Validar se o tabuleiro inicial é consistente.
- Resolver o puzzle automaticamente.
- Mostrar o processo de resolução.

Interface de Utilizador

A interface deve:

- Mostrar uma grelha 9x9 editável.
- Destacar células preenchidas pelo utilizador e células calculadas pelo algoritmo.
- Mostrar mensagens de erro para entradas inválidas.
- Permitir visualizar a resolução passo a passo.

Requisitos

- Implementar comparação entre as variantes de backtracking.
- Mostrar número de tentativas, retrocessos e tempo de execução.
- Permitir seleccionar o nível de dificuldade do puzzle.
- Detectar puzzles sem solução ou com múltiplas soluções.

Saídas esperadas

- Tabuleiro resolvido.
 - Número de retrocessos.
 - Tempo de execução.
 - Número de estados testados.
 - Comparação entre estratégias.
-
- Lista de conflitos resolvidos ou pendentes.
 - Tempo de execução.
 - Número de estados explorados.
 - Pontuação da solução.

4.2.11 Navegação de Personagem em Jogo 2D

Descrição do Problema

Criar uma aplicação que simule um personagem num jogo 2D que deve deslocar-se de uma posição inicial até um objectivo, evitando obstáculos, inimigos e armadilhas. O cenário deve ser modelado como um mapa em grelha.

Modelação esperada

Os estudantes devem definir:

- Estado: posição actual do personagem.
- Acções: mover para cima, baixo, esquerda, direita e, opcionalmente, diagonal.
- Custos: terreno normal, terreno difícil, armadilhas, proximidade de inimigos.
- Objectivo: alcançar o destino com menor custo possível.

Requisitos Funcionais

O sistema deve permitir:

- Criar ou carregar mapas.
- Definir obstáculos, inimigos, armadilhas e destino.
- Escolher algoritmo de busca.
- Visualizar o caminho escolhido.

Interface de Utilizador

A interface deve:

- Mostrar o mapa do jogo.
- Permitir editar obstáculos e perigos.
- Mostrar a exploração dos nós.
- Animar o movimento do personagem.

Requisitos avançados

- Inimigos com movimento simples.
- Recalcular rota quando o ambiente muda.
- Diferentes tipos de terreno com custos distintos.
- Comparar caminho mais curto com caminho mais seguro.

Saídas esperadas

- Caminho encontrado.
- Custo total do percurso.
- Nós explorados.
- Tempo de execução.

- Comparação entre algoritmos.

4.2.12 Transporte de Pacientes em Ambiente Hospitalar

Descrição do Problema

Desenvolver um sistema que planeie o transporte de pacientes dentro de um hospital, por exemplo entre enfermarias, salas de exames, bloco operatório e recepção. O sistema deve encontrar rotas eficientes, respeitando prioridades e restrições de circulação.

Modelação esperada

Os estudantes devem definir:

- Estado: localização actual do paciente ou transportador.
- Acções: deslocar-se entre sectores do hospital.
- Custos: distância, tempo, prioridade clínica e acessibilidade.
- Objectivo: transportar o paciente até ao destino correcto.

Requisitos Funcionais

O sistema deve permitir:

- Definir planta simplificada do hospital.
- Inserir pacientes, destinos e prioridades.
- Definir corredores bloqueados ou áreas restritas.
- Calcular rotas para um ou vários pacientes.

Interface de Utilizador

A interface deve:

- Mostrar a planta do hospital.
- Permitir seleccionar origem, destino e prioridade.
- Mostrar a rota calculada.
- Apresentar alertas para áreas bloqueadas.

Requisitos

- Gerir múltiplos pacientes.
- Priorizar pacientes urgentes.
- Recalcular rotas quando um corredor fica indisponível.
- Considerar elevadores ou escadas com custos diferentes.

Saídas esperadas

- Rota recomendada.
- Custo total.
- Tempo estimado.
- Nós explorados.
- Justificação da escolha da rota.

4.2.13 Problema dos Jarros de Água Generalizado

Descrição do Problema

Criar uma aplicação que resolva o problema dos jarros de água numa versão generalizada. Em vez de apenas dois jarros, o sistema deve permitir vários jarros com capacidades diferentes e objectivos configuráveis.

Modelação esperada

Os estudantes devem definir:

- Estado: quantidade de água em cada jarro.
- Acções:
 - Encher jarro.
 - Esvaziar jarro.
 - Transferir água entre jarros.
- Teste de objectivo: atingir uma quantidade desejada em pelo menos um jarro ou numa combinação de jarros.
- Custo: número de operações ou volume total movimentado.

Requisitos Funcionais

O sistema deve permitir:

- Definir número de jarros.
- Definir capacidades.
- Definir quantidade objectivo.
- Escolher algoritmo de busca.
- Mostrar a sequência de operações.

Interface de Utilizador

A interface deve:

- Mostrar os jarros visualmente.
- Permitir configurar capacidades e objectivo.

- Mostrar cada operação executada.
- Apresentar o estado dos jarros em cada passo.

Requisitos avançados

- Suporte a três ou mais jarros.
- Comparar solução com menor número de passos e menor volume movimentado.
- Detectar automaticamente objectivos impossíveis.
- Mostrar grafo ou árvore parcial de busca.

Saídas esperadas

- Sequência de operações.
- Número de passos.
- Volume total movimentado.
- Nós explorados.
- Tempo de execução.

4.2.14 Planeamento de Missão de Resgate

Descrição do Problema

Desenvolver uma simulação de uma missão de resgate em que uma equipa deve localizar vítimas, evitar zonas perigosas e regressar à base. O ambiente pode representar uma cidade, floresta, edifício destruído ou zona afectada por desastre.

Modelação esperada

Os estudantes devem definir:

- Estado: posição da equipa, vítimas já resgatadas, recursos disponíveis e nível de risco.
- Acções: mover, resgatar vítima, regressar à base.
- Custos: distância, risco, consumo de recursos e prioridade da vítima.
- Objectivo: resgatar vítimas e regressar à base com custo aceitável.

Requisitos Funcionais

O sistema deve permitir:

- Criar um mapa com obstáculos e zonas de risco.
- Definir localização da base.
- Definir localização e prioridade das vítimas.
- Escolher algoritmo de busca.
- Executar a missão simulada.

Interface de Utilizador

A interface deve:

- Mostrar o mapa da missão.
- Destacar base, vítimas, obstáculos e zonas perigosas.
- Mostrar a rota da equipa.
- Apresentar o progresso da missão.

Requisitos

- Múltiplas vítimas com prioridades diferentes.
- Recursos limitados, como energia ou tempo.
- Replaneamento quando surge novo obstáculo.
- Comparar estratégia mais curta, mais segura e mais eficiente.

Saídas esperadas

- Sequência da missão.
- Vítimas resgatadas.
- Custo total.
- Risco acumulado.
- Nós explorados.
- Tempo de execução.

4.3 Requisitos gerais

Cada projecto deve incluir:

- Pelo menos **3 algoritmos de busca**, incluindo pelo menos um algoritmo informado.
- Interface de utilizador funcional.
- Visualização do caminho ou sequência de estados.
- Contagem de nós expandidos.
- Tempo de execução.
- Custo da solução.
- Comparação entre algoritmos.
- Relatório explicando a modelação do problema.

5 Critérios de avaliação

Critério	Peso
Modelação correcta do problema de busca	20%
Implementação dos algoritmos	25%

Interface e visualização	20%
Comparação experimental dos algoritmos	15%
Relatório técnico	10%
Apresentação e defesa	10%